

Arrays e Funções Úteis em PHP

SUMÁRIO

1. [Introdução aos Arrays no PHP](#)
2. [Declaração Dinâmica de Arrays](#)
3. [Tipos de Arrays](#)
4. [Funções Nativas para Manipulação de Arrays](#)
5. [Impressão e Iteração de Arrays](#)
6. [Funções Diversas de Validação](#)
7. [Validação de E-mail com filter_var](#)
8. [Funções de Data e Hora](#)
9. [Validação de Datas com checkdate](#)
10. [Funções de Manipulação de Strings](#)
11. [Funções Matemáticas](#)
12. [Funções Avançadas para Arrays](#)
13. [Conversão entre String e Array: implode\(\) e explode\(\)](#)
14. [Notas de Atualização para PHP 8+](#)

1. INTRODUÇÃO AOS ARRAYS NO PHP

Array é uma variável unidimensional que aceita mais de um valor armazenado por índice.

No PHP, podemos utilizar arrays de:

- **Literais (Strings)**
- **Numéricos** (inteiros e fracionários)
- **Lógicos** (booleanos)
- **Homogêneos/Mistos** (todos os tipos misturados)

Característica Importante do PHP:

No PHP, podemos declarar arrays (vetores) **dinamicamente**, ou seja, **não precisamos declarar o tamanho ou tipo previamente**. Basta inserirmos valores dentro de uma variável e o PHP entenderá automaticamente que aquela variável é um array.

Boa prática: Embora opcional, é aconselhável informar explicitamente que a variável será um array, para melhorar a legibilidade e evitar erros.

2. DECLARAÇÃO DINÂMICA DE ARRAYS

Sintaxe Tradicional (funciona, mas menos moderna):

```
<?php  
$x = array(); // Declaração explícita de array vazio
```

Sintaxe Moderna (recomendada PHP 5.4+ e PHP 8):

```
<?php  
$x = []; // Declaração curta e moderna
```

Inserindo valores nas posições do array:

Utilizamos os colchetes [] após o nome do array para adicionar valores sequencialmente:

```
<?php  
$x = []; // ou array()  
$x[] = 'Roberto';    // índice 0  
$x[] = 'Vicentino';  // índice 1  
$x[] = 'Thomaz';     // índice 2  
?>
```

PHP 8: Ambos os métodos funcionam, mas [] é mais performático e é o padrão da comunidade moderna.

3. TIPOS DE ARRAYS

3.1 Array de Strings

Quando inserimos apenas valores entre aspas (" " ou ' '), o PHP cria um array de strings:

```
<?php  
$nome = [];  
$nome[] = "Roberto";  
$nome[] = "Manoel";  
$nome[] = "Jozicréusom";  
?>
```

3.2 Array de Números (inteiros e fracionários)

Quando inserimos apenas valores numéricos **sem aspas**, o PHP cria um array numérico:

```
<?php
$nota = [];
$nota[] = 10; // inteiro
$nota[] = 9.5; // float
$nota[] = 8.5; // float
?>
```

3.3 Array Homogêneo/Misto (Números e Strings)

O PHP permite **liberamente** a utilização de arrays com tipos misturados:

```
<?php
$misc = [];
$misc[] = "Roberto"; // string
$misc[] = 27; // inteiro
$misc[] = 8.5; // float
```

3.4 Array Associativo (Chaves Personalizadas)

Utilizamos chaves nomeadas em vez de índices numéricos sequenciais:

```
<?php
$a = [];
$a["nome"] = "Roberto";
$a["idade"] = 56;
$a["telefone"] = "91499128";
$a["email"] = "eng.cralves@gmail.com";
$a["cidade"] = "Poa";
?>
```

PHP 8 Tipagem Opcional: Você pode declarar tipos estritos para arrays:

```
function processar(array $dados): void { ... }
function somarNotas(array $notas): float { ... }
```

4. FUNÇÕES NATIVAS PARA MANIPULAÇÃO DE ARRAYS

Função	Descrição	Perde Associação?	Exemplo
<code>`sort(\$array)`</code>	Ordena pelos valores em ordem crescente	Sim	<code>`sort(\$notas);`</code>
<code>`rsort(\$array)`</code>	Ordena pelos valores em ordem decrescente	Sim	<code>`rsort(\$notas);`</code>
<code>`asort(\$array)`</code>	Ordena pelos valores , mantém associação das chaves	Não	<code>`asort(\$pessoa);`</code>
<code>`ksort(\$array)`</code>	Ordena pelas chaves associativas em ordem crescente	Não	<code>`ksort(\$pessoa);`</code>
<code>`krsort(\$array)`</code>	Ordena pelas chaves em ordem decrescente	Não	<code>`krsort(\$pessoa);`</code>
<code>`arsort(\$array)`</code>	Ordena pelos valores em ordem decrescente, mantendo associação	Não	<code>`arsort(\$notas);`</code>
<code>`shuffle(\$array)`</code>	Embaralha aleatoriamente os elementos do array	Sim (reindexa)	<code>`shuffle(\$nomes);`</code>
<code>`count(\$array)`</code>	Retorna a quantidade de elementos do array	-	<code>`count(\$nomes);`</code>
<code>`sizeof(\$array)`</code>	Alias de <code>`count()`</code> - mesma funcionalidade	-	<code>`sizeof(\$nomes);`</code>
<code>`array_sum(\$array)`</code>	Soma todos os valores numéricos do array	-	<code>`array_sum(\$notas);`</code>
<code>`array_keys(\$array)`</code>	Retorna um array apenas com as chaves	-	<code>`array_keys(\$a);`</code>
<code>`array_values(\$array)`</code>	Retorna um array apenas com os valores	-	<code>`array_values(\$a);`</code>
<code>`range(\$inicio, \$fim)`</code>	Gera um array com sequência de valores	-	<code>`range(1, 10);`</code>

Exemplos Práticos:

```
<?php
```

```
// Ordenação que PERDE associação (arrays indexados)
$notas = [8.5, 10, 7.0, 9.5];
sort($notas); // [7.0, 8.5, 9.5, 10]

// Ordenação que MANTÉM associação (arrays associativos)
$alunos = ["Ana" => 9.5, "Bruno" => 7.0, "Carla" => 10];
asort($alunos); // Mantém: Bruno=>7.0, Ana=>9.5, Carla=>10

// Embaralhando
$scores = ["vermelho", "azul", "verde", "amarelo"];
shuffle($scores); // Ordem aleatória

// Contando elementos
echo count($scores); // 4

// Somando valores numéricos
$valores = [10, 20, 30.5];
echo array_sum($valores); // 60.5

// Gerando sequências
$numeros = range(1, 10); // [1,2,3,4,5,6,7,8,9,10]
$letras = range('A', 'Z'); // ['A','B','C',...'Z']
$passos = range(0, 100, 10); // [0,10,20,30,...,100] - com passo
?>
```

5. IMPRESSÃO E ITERAÇÃO DE ARRAYS

Array de Exemplo para Demonstração:

```
<?php
$nomes = [];
$nomes[] = "Roberto Alvarez";
$nomes[] = "Jozinéusom";
$nomes[] = "Euricléia Mattos";
$nomes[] = "Romersindo Fulviano";
?>
```

Apenas para Teste/Debug (NÃO usar em produção):

```
<?php
// Exibe estrutura do array de forma legível
print_r($nomes);

// Exibe informações detalhadas: tipo, tamanho, valor de cada posição
var_dump($nomes);

// Exemplo de saída do var_dump:
// array(4) { [0]=> string(11) "Roberto Alvarez" [1]=> string(11) "Jozinéusom" ... }
?>
```

Iteração em Produção (Maneiras Corretas):

FOREACH – MAIS RECOMENDADA! (Simples e segura)

```
<?php
foreach ($nomes as $nomeAux) {
    echo $nomeAux . "<br>";
}
?>
```

FOREACH com Chave e Valor (para arrays associativos)

```
<?php
$ pessoa = ["nome" => "Roberto", "idade" => 56, "cidade" => "Poa"];

foreach ($pessoa as $chave => $valor) {
    echo "$chave: $valor<br>";
}

// Saída: nome: Roberto<br>idade: 56<br>cidade: Poá<br>
?>
```

WHILE (controle manual do índice)

```
<?php
$i = 0;
while ($i < count($nomes)) {
```

```

        echo $nomes[$i] . "<br>";
        $i++;
    }
?>

```

FOR (estrutura clássica)

```

<?php
for ($i = 0; $i < count($nomes); $i++) {
    echo $nomes[$i] . "<br>";
}
?>

```

Dica PHP 8: Para evitar chamadas repetidas a count() dentro do loop (que pode impactar performance em arrays grandes), armazene o resultado antes:

```

$total = count($nomes);
for ($i = 0; $i < $total; $i++) { ... }

```

6. FUNÇÕES DIVERSAS DE VALIDAÇÃO

Função	Descrição	Retorno	Exemplo
<code>`is_numeric(\$valor)`</code>	Testa se o valor é número (inteiro, float ou string numérica)	<code>`true`/`false`</code>	<code>`is_numeric("123")` → `true`</code>
<code>`!is_numeric(\$valor)`</code>	Testa se o valor **NÃO** é número	<code>`true`/`false`</code>	<code>`!is_numeric("abc")` → `true`</code>
<code>`is_null(\$variavel)`</code>	Testa se a variável é <code>`null`</code> (não contém valor)	<code>`true`/`false`</code>	<code>`is_null(\$x)`</code>
<code>`isset(\$variavel)`</code>	Verifica se a variável **existe** e **não é null**	<code>`true`/`false`</code>	<code>`isset(\$nome)`</code>
<code>`empty(\$variavel)`</code>	Verifica se a variável está vazia (<code>`""`</code> , <code>`0`</code> , <code>`null`</code> , <code>`[]`</code> , <code>`false`</code>)	<code>`true`/`false`</code>	<code>`empty(\$texto)`</code>
<code>`unset(\$variavel)`</code>	Remove/destrói a variável da memória	<code>`void`</code>	<code>`unset(\$temp)`</code>

Exemplos Práticos:

```
<?php
// is_numeric
$valor1 = "123";
$valor2 = "abc";

if (is_numeric($valor1)) {
    echo "$valor1 é numérico<br>"; // Será exibido
}

if (!is_numeric($valor2)) {
    echo "$valor2 NÃO é numérico<br>"; // Será exibido
}

// is_null e isset
$x = null;
$y = "conteúdo";

var_dump(is_null($x)); // bool(true)
var_dump(isset($x)); // bool(false) - null não é considerado "setado"
var_dump(isset($y)); // bool(true)

// empty
$vazia = "";
$zero = 0;
$falso = false;

var_dump(empty($vazia)); // true
var_dump(empty($zero)); // true (cuidado: 0 é considerado "vazio!")
var_dump(empty($falso)); // true

// unset
$temp = "valor temporário";
echo isset($temp) ? "Existe<br>" : "Não existe<br>"; // Existe
```



```
unset($temp);  
echo isset($temp) ? "Existe<br>" : "Não existe<br>"; // Não existe  
?>
```

Atenção PHP 8: `is_countable()` foi introduzida para verificar se uma variável pode ser usada com `count()`, evitando warnings:

```
if (is_countable($dados) && count($dados) > 0) { ... }
```

7. VALIDAÇÃO DE E-MAIL COM `filter_var`

Podemos validar e-mails utilizando a função nativa `filter_var()` com o filtro `FILTER_VALIDATE_EMAIL`.

Sintaxe:

```
filter_var($email, FILTER_VALIDATE_EMAIL)
```

Comportamento:

- Retorna o **e-mail validado** (string) se for válido
- Retorna false se for inválido

Exemplo Completo:

```
<?php  
$email = "eng.cralves@gmail.com";  
  
// Forma simplificada (funciona em if/else)  
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    echo 'E-mail válido!';  
} else {  
    echo 'E-mail inválido!';  
}  
  
// Forma estrita (recomendada para atribuição)  
$emailValidado = filter_var($email, FILTER_VALIDATE_EMAIL);  
if ($emailValidado !== false) {  
    echo "E-mail confirmado: $emailValidado";  
}
```

```
}  
?>
```

Testando Múltiplos E-mails:

```
<?php  
$emails = [  
    "usuario@dominio.com",    // válido  
    "nome.sobrenome@empresa.com.br", // válido  
    "invalido@",              // inválido  
    "@dominio.com",           // inválido  
    "sem-arroba.com"          // inválido  
];  
  
foreach ($emails as $teste) {  
    $resultado = filter_var($teste, FILTER_VALIDATE_EMAIL) ? "✓ Válido" : "X  
Inválido";  
    echo "$teste → $resultado<br>";  
}  
?>
```

PHP 8: `filter_var()` mantém compatibilidade total. Novos filtros foram adicionados para URL, IP, UUID, etc.

8. FUNÇÕES DE DATA E HORA

Configurando Fuso Horário (OBRIGATÓRIO no PHP 8+):

```
<?php  
// Definir fuso horário para evitar warnings e garantir precisão  
date_default_timezone_set('America/Sao_Paulo');  
?>
```

Buscando Data e Hora do Sistema:

```
<?php  
// Data atual no formato ISO: YYYY-MM-DD
```

```
echo date('Y-m-d'); // Ex: 2026-05-06
```

```
// Hora atual no formato 24h: HH:MM
```

```
echo date('G:i'); // Ex: 14:30
```

```
// Data e hora completas
```

```
echo date('d/m/Y H:i:s'); // Ex: 06/05/2026 14:30:45
```

```
// Dia da semana por extenso
```

```
echo date('l'); // Ex: Wednesday
```

```
// Mês por extenso
```

```
echo date('F'); // Ex: May
```

```
?>
```

Principais Formatos de date():

Formato	Descrição	Exemplo de Saída
`Y`	Ano com 4 dígitos	2026
`y`	Ano com 2 dígitos	26
`m`	Mês com 2 dígitos (01-12)	05
`n`	Mês sem zero à esquerda (1-12)	5
`d`	Dia com 2 dígitos (01-31)	06
`j`	Dia sem zero à esquerda (1-31)	6
`H`	Hora 24h com 2 dígitos (00-23)	14
`G`	Hora 24h sem zero à esquerda (0-23)	14
`i`	Minutos com 2 dígitos (00-59)	30
`s`	Segundos com 2 dígitos (00-59)	45
`l` (L minúsculo)	Dia da semana por extenso	Wednesday
`D`	Dia da semana abreviado	Wed
`F`	Mês por extenso	May
`M`	Mês abreviado	May

9. VALIDAÇÃO DE DATAS COM checkdate

A função `checkdate()` valida se uma data é **calendariamente correta**, recebendo três parâmetros: **mês, dia, ano**.

Sintaxe:

`checkdate(int $month, int $day, int $year): bool`

Exemplo Prático:

```
<?php
$mes = 10;
$dia = 5;
$ano = 2012;

if (checkdate($mes, $dia, $ano)) {
    echo "Data válida!"; // 05/10/2012 existe
} else {
    echo "Data inválida!";
}

// Testando datas inválidas:

var_dump(checkdate(2, 30, 2026)); // false - fevereiro não tem 30 dias
var_dump(checkdate(13, 1, 2026)); // false - mês 13 não existe
var_dump(checkdate(4, 31, 2026)); // false - abril tem apenas 30 dias
var_dump(checkdate(2, 29, 2024)); // true - 2024 é bissexto!
var_dump(checkdate(2, 29, 2026)); // false - 2026 NÃO é bissexto
?>
```

Dica: `checkdate()` considera automaticamente anos bissextos!

10. FUNÇÕES DE MANIPULAÇÃO DE STRINGS

Função	Descrição	Exemplo	Resultado
<code>`strtolower(\$str)`</code>	Converte para minúsculas	<code>`strtolower("PHP")`</code>	<code>`"php"`</code>
<code>`strtoupper(\$str)`</code>	Converte para maiúsculas	<code>`strtoupper("php")`</code>	<code>`"PHP"`</code>
<code>`strlen(\$str)`</code>	Conta caracteres (bytes)	<code>`strlen("Olá")`</code>	<code>`4`</code>
<code>`mb_strlen(\$str, 'UTF-8')`</code>	Conta caracteres em UTF-8	<code>`mb_strlen("Olá", 'UTF-8')`</code>	<code>`3`</code>
<code>`str_word_count(\$str)`</code>	Conta palavras	<code>`str_word_count("Olá mundo")`</code>	<code>`2`</code>
<code>`ucwords(\$str)`</code>	Primeira letra de cada palavra em maiúscula	<code>`ucwords("ola mundo")`</code>	<code>`"Ola Mundo"`</code>
<code>`ucfirst(\$str)`</code>	Primeira letra da string em maiúscula	<code>`ucfirst("ola")`</code>	<code>`"Ola"`</code>
<code>`lcfirst(\$str)`</code>	Primeira letra da string em minúscula	<code>`lcfirst("OLA")`</code>	<code>`"oLA"`</code>
<code>`trim(\$str)`</code>	Remove espaços do início e fim	<code>`trim(" texto ")`</code>	<code>`"texto"`</code>
<code>`ltrim(\$str)`</code>	Remove espaços do início	<code>`ltrim(" texto")`</code>	<code>`"texto"`</code>
<code>`rtrim(\$str)`</code>	Remove espaços do fim	<code>`rtrim("texto ")`</code>	<code>`"texto"`</code>
<code>`str_replace(\$busca, \$troca, \$str)`</code>	Substitui texto	<code>`str_replace("a", "@", "casa")`</code>	<code>`"c@s@"`</code>

Atenção com strlen() e UTF-8:

strlen() conta **bytes**, não caracteres. Para strings com acentos/emojis, use mb_strlen() com encoding UTF-8.

Exemplos Completos:

```
<?php
$texto = " Linguagem de Programação em PHP ";
```

```

echo strtolower($texto); // " linguagem de programação em php "
echo strtoupper($texto); // " LINGUAGEM DE PROGRAMAÇÃO EM PHP "
echo strlen($texto); // 37 (conta bytes, incluindo espaços)
echo mb_strlen($texto, 'UTF-8'); // 35 (conta caracteres reais)
echo str_word_count($texto); // 5 palavras
echo ucwords($texto); // " Linguagem De Programação Em Php "
echo trim($texto); // "Linguagem de Programação em PHP"

// Combinação útil: limpar e formatar título
$titulo = trim(ucwords(strtolower($texto)));
echo $titulo; // "Linguagem De Programação Em Php"
?>

```

PHP 8 Novidades: Funções mais intuitivas para strings:

```

str_contains("Hello World", "World"); // true (substitui strpos() !== false)
str_starts_with("Hello", "He"); // true
str_ends_with("Hello", "lo"); // true

```

11. FUNÇÕES MATEMÁTICAS

Função	Descrição	Sintaxe	Exemplo	Resultado
<code>`pow(\$base, \$exp)`</code>	Potenciação	<code>`pow(2, 3)`</code>	2^3	<code>`8`</code>
<code>`sqrt(\$valor)`</code>	Raiz quadrada	<code>`sqrt(16)`</code>	$\sqrt{16}$	<code>`4`</code>
<code>`abs(\$valor)`</code>	Valor absoluto	<code>`abs(-10)`</code>	$ -10 $	<code>`10`</code>
<code>`round(\$valor, \$precisao)`</code>	Arredondamento	<code>`round(3.14159, 2)`</code>	3.14	<code>`3.14`</code>
<code>`ceil(\$valor)`</code>	Arredonda para **cima**	<code>`ceil(3.1)`</code>	$\rightarrow 4$	<code>`4`</code>
<code>`floor(\$valor)`</code>	Arredonda para **baixo**	<code>`floor(3.9)`</code>	$\rightarrow 3$	<code>`3`</code>
<code>`rand(\$min, \$max)`</code>	Número aleatório (legado)	<code>`rand(1, 10)`</code>	Aleatório	<code>`1-10`</code>
<code>`random_int(\$min, \$max)`</code>	Número aleatório **criptograficamente seguro**	<code>`random_int(1, 10)`</code>	Aleatório seguro	<code>`1-10`</code>

Exemplos Práticos:

```
<?php
// Potência: formas diferentes de fazer a mesma coisa
echo pow(2, 3); // 8 (função tradicional)
echo 2 ** 3;    // 8 (operador PHP 5.6+, recomendado)

// Raiz quadrada
echo sqrt(25); // 5
echo sqrt(2);  // 1.4142135623731

// Arredondamentos
$nota = 7.856;
echo round($nota, 2); // 7.86
echo ceil($nota);     // 8 (sempre para cima)
echo floor($nota);    // 7 (sempre para baixo)

// Aleatórios
echo rand(1, 6);      // Dado simples (não seguro para segurança)
echo random_int(1, 100); // Token, senha, sorteio seguro

// Valor absoluto
$saldo = -150.50;
echo "Débito de R$ " . abs($saldo); // Débito de R$ 150.5
?>
```

12. FUNÇÕES AVANÇADAS PARA ARRAYS

Array de Exemplo para Demonstração:

```
<?php
$a = [];
$a["nome"] = "Roberto";
$a["idade"] = 56;
$a["telefone"] = "91499128";
$a["email"] = "eng.cralves@gmail.com";
$a["cidade"] = "Poá";
```

?>

Impressão para Debug:

```
<?php
// Estrutura legível
print_r($a);

// Detalhes completos (tipo, tamanho, valor)
var_dump($a);

// Apenas as chaves
var_dump(array_keys($a));
// Saída: array(5) { [0]=> string(4) "nome" [1]=> string(5) "idade" ... }

// Apenas os valores
var_dump(array_values($a));
// Saída: array(5) { [0]=> string(7) "Roberto" [1]=> int(56) ... }
?>
```

Verificando Tamanho:

```
<?php
// Duas formas equivalentes:
echo count($a); // 5
echo sizeof($a); // 5 (alias de count)

// Dica PHP 8: verifique se é contável antes de usar count()
if (is_countable($a) && count($a) > 0) {
    echo "Array possui " . count($a) . " elementos";
}
?>
```

Soma de Valores Numéricos:


```
<?php
$notas = [8.5, 10, 7.0, 9.5];
echo "Média: " . array_sum($notas) / count($notas); // 8.75

// Array misto: apenas valores numéricos são somados
$misto = ["a", 10, "b", 20.5];
echo array_sum($misto); // 30.5 (ignora strings não-numéricas)
?>
```

13. CONVERSÃO ENTRE STRING E ARRAY: implode() E explode()

implode() – Junta elementos de um array em uma única string

Sintaxe: implode(string \$separator, array \$array): string

Ordem dos parâmetros: Primeiro o separador, depois o array (padrão desde PHP 4).

Exemplos:

```
<?php
// Juntando nome completo
$partesNome = ["Roberto", "Gonçalves", "Alvarez"];
$nomeCompleto = implode(" ", $partesNome);
echo $nomeCompleto; // "Roberto Gonçalves Alvarez"

// Com separador personalizado
$frutas = ["maçã", "banana", "laranja"];
echo implode(", ", $frutas); // "maçã, banana, laranja"
echo implode("###", $frutas); // "maçã###banana###laranja"

// Criando data formatada
$d = 10; $m = 7; $ano = 2012;
$dataArray = [$d, $m, $ano];
$dataFormatada = implode("/", $dataArray);
echo "Data: $dataFormatada"; // "Data: 10/7/2012"
```

```
// Exemplo divertido: formando palavra
$letras = ["b", "t", "t"];
echo implode("a", $letras); // "batata" 😊
?>
```

explode() – Divide uma string em um array (INVERSO do implode)

Sintaxe: explode(string \$separator, string \$string, int \$limit = PHP_INT_MAX): array

Exemplos:

```
<?php
// Separando nome completo
$nomeCompleto = "Roberto Gonçalves Alvarez";
$partes = explode(" ", $nomeCompleto);
print_r($partes);
// Array ( [0] => Roberto [1] => Gonçalves [2] => Alvarez )

// Separando data
$data = "10/7/2012";
$partesData = explode("/", $data);
echo "Dia: {$partesData[0]}, Mês: {$partesData[1]}, Ano: {$partesData[2]}";
// Dia: 10, Mês: 7, Ano: 2012

// Limitando número de divisões
$texto = "um|dois|três|quatro";
print_r(explode("|", $texto, 3));
// Array ( [0] => um [1] => dois [2] => três|quatro )
?>
```

Ciclo Completo: String → Array → String

```
<?php
$original = "PHP,JavaScript,Python,Rust";

// String para Array
```

```
$linguagens = explode(",", $original);

// Manipular o array
$linguagens[] = "Go"; // Adicionar nova linguagem
sort($linguagens); // Ordenar alfabeticamente

// Array para String
$atualizado = implode(" | ", $linguagens);

echo $atualizado;
// "Go | JavaScript | PHP | Python | Rust"
?>
```

14. NOTAS DE ATUALIZAÇÃO PARA PHP 8+

Compatibilidade Garantida

Todas as funções apresentadas neste material **funcionam perfeitamente no PHP 8+**, pois são funções nativas estáveis há várias versões.

Mudanças e Melhorias no PHP 8:

1. Sintaxe de Arrays

```
// Ambos funcionam, mas prefira a sintaxe curta:
$antigo = array("a", "b", "c"); // Funciona
$moderno = ["a", "b", "c"]; // Recomendado
```

2. count() com Variáveis Não-Contáveis

No PHP 8, passar uma variável que não é array/Countable para count() gera **Warning**:

```
// Pode gerar warning no PHP 8:
```

```
$valor = "texto";
echo count($valor); // Warning: count(): Parameter #1 must be of type Countable|array
```

```
// Forma segura:
```

```
if (is_countable($valor)) {  
    echo count($valor);  
} else {  
    echo 0; // ou trate o erro  
}
```

3. Tipagem de Arrays (Type Declarations)

PHP 8 permite declarar tipos estritos para parâmetros e retornos:

```
function processarNomes(array $nomes): void {  
    foreach ($nomes as $nome) {  
        echo htmlspecialchars($nome) . "<br>";  
    }  
}
```

```
function calcularMedia(array $notas): float {  
    return array_sum($notas) / count($notas);  
}
```

4. Novas Funções de String (PHP 8+)

Substituem padrões antigos com código mais legível:

// Antes (PHP 7 e anterior):

```
if (strpos($texto, "palavra") !== false) { ... }
```

// Agora (PHP 8+):

```
if (str_contains($texto, "palavra")) { ... }
```

// Outras novas:

```
str_starts_with($url, "https://");
```

```
str_ends_with($arquivo, ".pdf");
```

5. filter_var() Retorno Mais Preciso

Tecnicamente, filter_var() retorna a string validada ou false, não um booleano puro:

// Funciona em if/else:

```
if (filter_var($email, FILTER_VALIDATE_EMAIL)) { ... }
```

```
// Para atribuição segura:
$emailValidado = filter_var($email, FILTER_VALIDATE_EMAIL);
if ($emailValidado !== false) {
    // Use $emailValidado (já sanitizado)
}
```

6. Fuso Horário Obrigatório

Sempre defina o timezone para evitar warnings e inconsistências:

```
// No início do script ou no php.ini:
date_default_timezone_set('America/Sao_Paulo');
```

RESUMO RÁPIDO – CHEAT SHEET

```
<?php
// ===== ARRAYS =====
$lista = [];           // Declaração moderna
$lista[] = "valor";    // Adicionar
sort($lista);          // Ordenar (perde associação)
asort($lista);         // Ordenar (mantém associação)
count($lista);         // Quantos elementos?
foreach ($lista as $item) { ... } // Iterar (recomendado)

// ===== VALIDAÇÃO =====
is_numeric($x);        // É número?
isset($var);          // Existe e não é null?
empty($var);           // Está "vazio"?
filter_var($email, FILTER_VALIDATE_EMAIL); // E-mail válido?

// ===== DATA =====
date_default_timezone_set('America/Sao_Paulo');
date('Y-m-d');          // Data atual
checkdate($m, $d, $y);  // Data é válida?
```

```
// ===== STRINGS =====  
strtolower($s); strtoupper($s); // Caixa baixa/alta  
strlen($s); mb_strlen($s, 'UTF-8'); // Tamanho  
trim($s); // Remove espaços  
str_contains($s, "texto"); // PHP 8: contém?
```

```
// ===== MATEMÁTICA =====  
pow(2, 3); // 8  
2 ** 3; // 8 (operador moderno)  
sqrt(16); // 4  
random_int(1, 100); // Aleatório seguro
```

```
// ===== CONVERSÃO =====  
implode(",", $array); // Array → String  
explode(",", $string); // String → Array  
?>
```

APÊNDICE: Lista Completa de Funções por Categoria

Arrays

array(), [], sort(), rsort(), asort(), arsort(), ksort(), krsort(), shuffle(), count(), sizeof(),
array_sum(), array_keys(), array_values(), range(), print_r(), var_dump(), is_countable()

Validação

is_numeric(), is_null(), isset(), empty(), is_countable(), filter_var(),
FILTER_VALIDATE_EMAIL

Data/Hora

date(), date_default_timezone_set(), checkdate(), time(), strtotime()

Strings

strtolower(), strtoupper(), strlen(), mb_strlen(), str_word_count(), ucwords(), ucfirst(),
lcfirst(), trim(), ltrim(), rtrim(), str_replace(), str_contains(), str_starts_with(),
str_ends_with()

Matemática

`pow()`, `**`, `sqrt()`, `abs()`, `round()`, `ceil()`, `floor()`, `rand()`, `random_int()`

Conversão

`implode()`, `explode()`, `str_split()`, `json_encode()`, `json_decode()`